

Programmation Fonctionnelle

TD n° 3 : Encore sur les listes

Exercice 1 : Écrire une fonction `choose : 'a list -> 'a` choisissant au hasard et renvoyant un élément d'une liste non vide. Vous pouvez utiliser la fonction prédéfinie `Random.int : int -> int`. Pour tout entier `n` l'application `Random.int n` calcule un entier choisi au hasard entre 0 inclus et `n` exclus.

Exemples :

- `choose [6; 5; 8; 7; 3] = 5`
- `choose ['e'; 'c'; 'a'; 'd'; 'b'] = 'd'`

Exercice 2 : Écrire une fonction `choose_elements : 'a list -> int -> 'a list` telle que `choose_elements l n` renvoie une liste formée de `n` éléments choisis au hasard à des positions distinctes dans la liste `l`.

Exemples :

- `choose_elements [3; 5; 6; 7; 8] 3 = [3; 7; 6]`
- `choose_elements ['a'; 'b'; 'c'; 'd'; 'e'] 3 = ['a'; 'e'; 'd']`

Exercice 3 : (difficile) Écrire une fonction `choose_sublist : 'a list -> int -> 'a list` telle que `choose_sublist l n` renvoie une sous-liste de `l` à `n` éléments choisis au hasard. (Contrairement à l'exercice précédent, il faut donc conserver l'ordre des éléments de la liste d'origine.)

Exemples :

- `choose_sublist [3; 5; 6; 7; 8] 3 = [3; 6; 7]`
- `choose_sublist ['a'; 'b'; 'c'; 'd'; 'e'] 3 = ['a'; 'b'; 'e']`

Les fonctions suivantes attendent des listes dont les éléments sont de n'importe quel type permettant l'utilisation des opérateurs de comparaison génériques (`<`, `<=`, `=`, `>=`, `>`). Elles pourront donc manipuler indifféremment des listes d'`int`, de `char`, etc.

Exercice 4 : Écrire une fonction `insert` qui étant donnée une liste supposée triée pour l'ordre strictement croissant et un élément `x` renvoie la liste obtenue en insérant `x` à la bonne place. Si `x` est déjà dans la liste, celle-ci sera renvoyée telle quelle.

Exemples :

- `insert 5 [1; 3; 8] = [1; 3; 5; 8]`
- `insert 'e' ['a'; 'c'; 'g'] = ['a'; 'c'; 'e'; 'g']`

Exercice 5 : En utilisant la fonction `insert`, écrire une fonction `sort` permettant de trier une '`a list` quelconque par ordre croissant, en fusionnant les doublons.

Exemple : `sort [7; 8; 5; 2; 8] = [2; 5; 7; 8]`

Indication : comment obtenir `[2; 5; 7; 8]` de 7 et de `sort [8; 5; 2; 8]` ?

Exercice 6 : Écrire une fonction `mem_sorted` se comportant comme la fonction `List.mem`, et s'évaluant en un nombre minimum d'étapes en supposant que la liste donnée en entrée est triée.

Exercice 7 : Écrire les opérations d'union et d'intersection (`union_sorted`, `inter_sorted`) de deux listes triées.

Exemples :

– `union_sorted` [1; 3; 5] [2; 5; 8] = [1; 2; 3; 5; 8]

– `inter_sorted` [1; 3; 5] [2; 5; 8] = [5]

Exercice 8 : Écrire une fonction `quicksort` qui implémente la méthode du tri rapide sur une liste. Cette méthode fonctionne comme suit pour trier une liste de taille `n` :

- si la liste est vide, on a fini ;
- si la liste ne contient qu'un seul élément, on a fini ;
- sinon, choisir le premier élément `a`, trier d'un côté tous les éléments inférieurs à `a`, d'un autre côté tous les éléments supérieurs à `a` et combiner les deux sous-listes triées en une seule liste triée.